Configuration management — keeping it all together

S M Thompson

Any project, programme or organisation, working in any environment with released information, managing its change, maintaining traceability, and ensuring results always meet expectations, needs configuration management. Software projects introduce additional complexities — multiple developers working on the same item at the same time, the need for compatibility with other products and systems, targeting releases for multiple platforms, and supporting multiple versions (for example development and released versions). This paper describes the configuration management processes that support and manage products through their entire life cycle as they change and evolve.

1. Introduction

1.1 So what is CM?

Configuration management (CM) is a widely used term, with an equally wide range of definitions. CM is not project management, although an element of it. It is not responsible for all aspects of a project or product, but has an influencing role upon them.

CM is based upon a formalisation of good business practices and techniques — an integration of the technical and administrative actions of identifying, documenting, controlling, reporting and validating the functional and physical characteristics of a product during its life cycle. While such activities are followed routinely by every effective manager, self-imposed discipline lacks the depth and uniformity of process, documentation and accountability required today by BT's complex projects.

CM is more than just stand-alone version control, more than just process control, more than just bookkeeping. Put simply, it is the art of keeping track of which items within a product have changed, how they have changed, and how they are combined. It is the 'who, what, when, why and how' of every change, system build and integration.

1.2 Scope of this paper

Within Systems Integration (SI), BT is primarily concerned with software systems, hence the bias of this paper towards the principles of software configuration management. However, configuration management is configuration management, whatever the environment.

This paper describes the everyday practices used within SI's Configuration Management Unit to support a number of BT's major software systems, and draws upon their experience in the provision of CM services and consultancy within BT.

1.3 A brief history of CM

The discipline of CM evolved as a result of management techniques employed in the US defence industry during the 1950s, where manufacturing problems (e.g. poor quality, wrong parts being ordered, parts not fitting) were constantly leading to excessive cost and time overruns.

Until the 1940s, the design and manufacture of each preceding era's products could quite feasibly be understood and managed by one engineer. By the end of that decade, complexity had increased beyond the point where relying on an individual's discipline was sufficient. A new method for managing the design and configuration of complex equipment was required. It was also critical to communicate this information, especially to compensate for jobs being started by one engineer and continued by others.

The 1950s saw the start of the space programmes, with the race for a successful rocket launch. Time was critical, resulting in frequent changes to resolve incompatibilities among suppliers' components. Following a successful launch, one buyer said, 'build me another'. However, the prototype had been expended and adequate records of part change histories and change accomplishments had not been kept. The technical publications did not reflect all the changes made, and a second successful launch, or even the production of an identical rocket could not be guaranteed [1].

In 1962 the US Air Force produced the first standard for configuration management (AFSCM 375-1), defining orderly processes for use through a product's life cycle. CM quickly gained acceptance as the 'key cog' in design, development, construction, testing, delivery and operation; it was the communicator and controller of design through evolution [2].

In the early 1970s the US Department of Defense (DoD) produced the first standards recognising computer hardware and, perhaps more significantly, software. Unlike the rockets of the space race, software prototypes are not expended during testing. However, they are still not a sufficient basis for further development due to the severe information loss that occurs during development and maintenance (for example, why changes were made to items, and the dependencies between items). To prevent this information loss, the traditional 'nuts and bolts' configuration management mandates strict processes for part and assembly identification, for control and auditing of changes, and status accounting (a history of what, when and why events occurred).

The first universal standard for software CM was eventually published in 1985 (DoD STD 2167), followed by standards including IEEE 1042-1987 and IEEE 828-1990. Today, the ISO 9000 series of standards demands the use of configuration management for software development. ISO 9000-3:1991 provides guidelines for the application of ISO 9001 to the supply and maintenance of software. It proposes controls and methods for developing software that meet user requirements, primarily by preventing non-conformity at all stages in the life cycle from development to maintenance. Major issues covered by this standard are quality systems, CM plans, documentation control, metrics, and software development procedures.

A relatively recent standard is ISO/IEC 12207-1995. It provides complete frameworks for the software life cycle, its management and improvement. This standard specifically identifies CM as a support process, and is a reference point for several (emerging) engineering disciplines, including programme management, quality assurance, verification and validation, and metrics.

1.4 So why is CM needed?

In today's business environment, software systems are critical in helping to provide BT's competitive edge. The market is demanding quality software in shorter time-scales. However, across the IT industry as a whole:

- the demand for information systems is often greater than the ability to supply,
- productivity in production is poor, especially in the field of software component reuse,
- delivered systems are of poor quality — a result of inadequate specifications causing an inability to measure conformance to standards,
- customers seldom get what they want, initially or after change, leading to a poor perception of the supplier.

1.4.1 Managing change and evolution

Software is developed through a process of constant change and evolution. Typically, system components are progressed and modified in isolation, only being drawn together as the final system is integrated or built. Components, even though developed in isolation, have dependencies. A minor change to one module may have unexpected and serious consequences for any number of others.

Without control, unauthorised (and therefore untestable) changes will occur without documentation, and without prior analysis of their impact and value. Authorised changes are often not much better, being inadequately documented and offering no option for being easily rolled back out of the product. When managed properly, change is a vehicle for optimising costs along with other interface and project control considerations. Ultimately, the absence of formal controls results in a state of technical anarchy.

1.4.2 Supporting variants

Deliveries of software systems may take several forms, known as 'variants', e.g. for different platforms, natural languages or for features unique to a particular customer. These systems are the products of teams, whose members have varying responsibilities, but their component dependencies mean they cannot work in isolation. Team members have different functional roles — some will be enhancing the product for future releases, while others will be supporting the current release. Multiple versions and variants, change dependencies, fault fixes and enhancements must all be managed. A controlled and consistent management structure is the only guarantee that high-quality systems can be built and subsequently maintained.

1.4.3 Encouraging and managing reuse

Released software is an asset that can be reused without change. Unfortunately, a lack of confidence in component specification and definition, plus a lack of knowledge of what is available, means the reuse of software components is generally low; but components must be reused, being tailored to customer needs, because their complex nature prohibits costly custom re-development.

To reuse, development must be controlled and then catalogued — there must be information about what the

components actually are and what they actually do. However, reusable components will still need to be maintained, increasing the complexity of managing change consistently across products that could be widely distributed.

1.5 CM activities

Most developers recognise the need to control versions. However, generally neither they nor their management realise the extent to which effective CM contributes to solving many of the problems faced in managing and controlling the development process. Configuration management offers solutions to the problems outlined above, and is recognised as a quick and efficient method of improving the quality of software development [3].

Configuration management is a collection of techniques used to control all the components of any product's infrastructure during their development and maintenance. The main activities covered in this paper are:

- configuration identification — identifying what is to be controlled,
- baselining — recording configurations,
- version control — managing the evolution of controlled components,
- change control and problem management — managing the evolution of controlled components,
- build management — creating products from the controlled components,
- release management — managing the release of integrated products and their subsequent life.

2. Configuration identification

2.1 How much control is needed?

The foundations of configuration management are in the control of individual, uniquely identifiable elements, known as 'configuration items' (CIs). The granularity of a CM system's CI definition determines the level of control it can provide. At one extreme, software systems could be the most basic CI. Alternatively, every procedure or line of code could be a CI. Both cases would be unmanageable, with control ranging from almost non-existent to overwhelming.

2.2 What should be controlled?

In software systems, the basic CIs are normally defined at the level of files or code modules. Of course, there is more to a software system than just code, e.g. documents (requirements, specifications, designs, test cases, release notes, user guides, etc), the required hardware components and their configurations, operating systems, commercially available products from third-party vendors, and run-time libraries.

While the basic CIs must be defined at a sufficiently low level to provide control, they may be used to derive more complex configurations. One CI may be transformed into another, and a CI may be composed of several others. For example, compiling a source file CI to an executable file CI, and defining product CIs to include executable software CIs plus documentation CIs respectively.

2.3 CI identification and storage

CIs have been defined as individual, uniquely identifiable items. Where those items are stored electronically, by definition their identifiers must be unique. Other CIs, especially physical items, are usually identified by serial or part numbers as used in the traditional nuts and bolts school of CM.

Besides a CI's name, its version, revision and, if appropriate, variant identifiers must also be recorded. A numbering scheme is usually employed for this purpose such as:

main version . *main revision* [. *variant*]

For example, 1.0 would be the first instance of a CI ever created, 1.1 the first revision, and 1.0.a the first variant of the original. Once identified, CIs along with details of how they are related must be deposited in a secure storage facility, preferably independent from the development environment. For physical items, records of part numbers and configurations are stored. Access restrictions must be applied to these libraries, although to what extent is dependent upon the library's contents and the product's maturity.

3. Baselining

3.1 What program is this?

After a software system is built or integrated, it passes into an integration testing phase, is subsequently released, and then comes under maintenance. Unfortunately, even by integration testing, a system's exact configuration can be in doubt, all too commonly raising the question: 'What program is this?'

The problem becomes more serious if access to code is not restricted. By the time testing detects problems, code can have changed beyond recognition. If there has been no control, the effort in testing will have been in vain; there is no (certain) way to reproduce the tested item or know what its component sources were. In the absence of a reference

point, change is meaningless, a critical point too often overlooked.

Should this problem arise in the maintenance of a released product, the consequences are even worse. In the past software has not been, and could not be, maintained for this very reason — entirely new products have had to be developed instead.

3.2 System reproduction

One objective of CM is to be able to reproduce at any time all or part of a system to a known past or present state. The ability to reconstruct configurations can be of particular use during early stages of the development life cycle when products are evolving rapidly, and is essential for future maintenance releases. Keeping explicit copies of everything that had existed is not practical, and would be difficult to manage. To permit the reproduction of a system, all changes to its CIs must be made against the libraries' CIs, and the libraries subsequently updated.

Baselines are applied to CI libraries to record their state at specified times, typically when builds or releases of the system are made. Baselines provide:

- roll-back points should the need to withdraw functionality or recover from disaster occur,
- a firm basis for future work.

In the absence of an agreed baseline, there may be conflict over the correct interpretation of changes. For example, where changes have been implemented at approximately the same time, taking a baseline ensures there is no dispute over product definition. Software systems do not evolve from code modules alone. The baseline should reflect this, instead of just being a record of source code. Requirements and design documentation, hardware, third-party software products and environments (variables, paths, operating systems, etc) all play a role in the development of systems. The baseline must record all these items, their revision level and status.

4. Version control

4.1 Managing CI enhancement and baselines

As already indicated, before a meaningful baseline can be defined, its constituent CIs must be uniquely identified, and stored so they can be accessed as they were when the baseline was taken. As electronic storage demands unique identification of items, ensuring CI names are unique is straightforward. However, version control, or controlling the revisions, versions and variants of CIs, is complex.

The difference between revisions, versions and variants is not always clear. Strictly, a new revision fixes faults, while a new version changes functionality. CI variants (commonly referred to as branches) are parallel developments from the same source, used, for example, where different natural languages or target platforms must be supported. The terms revision and version are often used interchangeably, which is generally acceptable as there is no difference in the way they are managed.

Some CM practitioners promote immutability where, whenever a CI is changed, its name is changed too. This very weak form of CM is not sufficient for version control, typically being implemented using only the development environment's file system. It can never guarantee that revisions contain only authorised changes and it cannot prevent unauthorised and unwanted modifications. More seriously, it is difficult — if not impossible — to detect changes, and there are no mechanisms to prevent either accidental or malicious CI corruption and loss. Such systems rely too heavily on the developers' integrity and assume they will always remember to update filenames every time a change is made, and in any case are unmanageable for more than a small number of revisions.

Others keep back-up tapes as a form of baseline; but how can they easily determine which CIs changed from one baseline to the next, and in which version of the CIs particular changes were introduced? Other development problems caused by a lack of version control include:

- the multiple maintenance problem — keeping multiple copies of identical software identical,
- the shared data problem — simultaneous access and modification of the same data,
- the simultaneous update problem — caused when two different changes are made to the same source, with the original source then being replaced by first one then the other of the modified items (a common consequence is the re-appearance of faults that had previously been fixed).

4.2 Software version-control tools

Most CM tools provide support for configuration identification and version control, allowing software development to be integrated directly with CM procedures. Naturally not just code but all items capable of being stored electronically may be version controlled by CM tools. For effective CM a version control system must meet, as a minimum, the following criteria.

- Manage access to, and control the sharing of, sources — in multi-user parallel development environments, teams must have visibility of each other's changes, and access restrictions are essential to prevent unauthorised access, access at inappropriate times and multiple accesses.

- Support parallel development, allowing easy creation of CI variants and, if appropriate, their later re-integration or merging — this support is essential where components are being developed by several people at the same time, or if bug fixes are required to maintain older released components.
- Store change histories — records must be kept of all changes applied to all CIs. The change history should show how and why a CI has evolved over time. It provides an audit trail, essential for all but the most trivial of projects.
- Allow the creation and management of baselines.

Utilities such as Unix's SCCS and RCS, and those CM tools whose version control is built on top of them, are not sufficient for version control in complex multi-user environments, as they have insufficient mechanisms for enforcing access restrictions.

4.3 CI version-control archive files

The principle behind version control in many tools is that of an archive file. A CI is stored (conceptually) as one item containing an ordered set of revisions and branches. The archive files are 'owned' by the version-control system, meaning its contents may only be accessed and updated through the version-control system's interface.

Figure 1 illustrates the concept of an archive file in its simplest form. It shows a CI called *hello_world.c* with three revisions, numbered as per the convention in section 2.3.

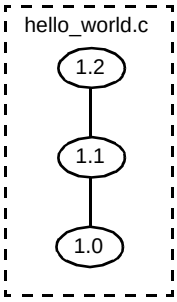

Fig 1 Logical structure of a version control archive file.

The storage of archive files varies between tools, but commonly they are found as items within a standard hierarchical file system. For storage efficiency, non-binary (e.g. ASCII) CIs may be stored using 'deltas' where only one revision is stored fully. This full version is usually the most recent one in the main development stream. The deltas specify the changes between revisions in terms of lines added, deleted and modified. Although the storage space required could be reduced considerably, for CIs with many revisions the time to access older or branch revisions can increase dramatically over a non-delta system.

To aid multi-user working and reuse, the version control system should be a centralised repository, typically divided into areas reserved for particular desired configurations or projects. In addition to aiding reuse, the version control system should be independent from all parts of the life cycle to enforce its use and ensure its integrity.

4.4 Creating new revisions

To update a CI revision, it must be checked out of the version control system into a separate external location. After modification, the revised CI is checked back into the library. As a minimum, checking out a CI revision places a lock on that revision preventing any other writable copies of the item being taken. Other revisions and read-only copies of locked CIs may still be obtained. Depending upon the tools and the product's maturity, other access controls may be in place. For example, early in the life cycle, the only restrictions necessary may be the enforcement of mutually exclusive access; later access to CIs may be restricted to individuals granted the correct permissions.

A change history should be associated with each new CI revision in the archive, providing an audit trail of why and when the CI was changed. Figure 2 illustrates how a version control system could logically associate change logs with CI revisions.

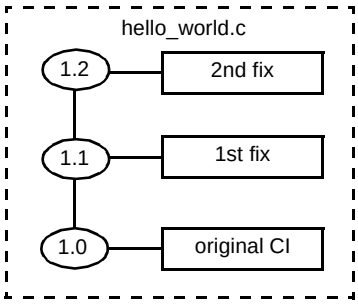

Fig 2 Associating change histories with versions in the archive file.

4.5 Variants

As the reader will have noticed, the term variant appears to have been used in the context of both basic CIs and of entire systems. The two are — or rather ought to be — tightly related.

Firstly, consider CI variants, as illustrated in Fig 3 (for clarity change logs are not shown in this or subsequent illustrations). Imagine revision 1.1 of *hello_world.c* is a component of a released system. After revision 1.2 had been created, a bug is found in 1.1. A new release fixing the bug is required, but without 1.2's changes. Locking out revision 1.1 for modification automatically creates a branch revision (1.1.a) when the CI is checked back into the system. In Fig 3, a further revision of the variant is also shown.

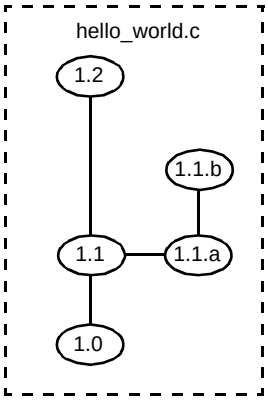

Fig 3 Creating a variant in an archive file.

System variants should almost always be built from CI variants. In practice, even where CM systems are in place, this does not always happen. Rather than creating variants in all the individual CI archives, a common alternative is to establish a copy of the baseline from which the variant system is to be derived.

The variant baseline is then created by adding, removing and changing some of the CIs. Most CIs remain common between the variant product and the original source. Why this approach should be taken is difficult to understand, but can only be contributed to a lack of understanding of variants and the consequences of not using them. Consider, for example, the implications of upgrading the common base platform if copies of CIs, not variants, have been used.

Even when done properly, managing system variants is one of the most cumbersome tasks in software configuration management. It is neither as well understood as version control, nor as well supported by tools, and a cause of the multiple-maintenance problem. Re-applying the same modification manually is tedious, and introduces a serious risk of creating a variety of further errors. Additionally, as products become more complex, and reuse increases, the risk of missing updates to any of the common files increases. The variants can easily get out of synchronisation, and the common CI is no longer common.

4.6 Merging variants

Variants for different platforms or natural languages will always remain as such. If variants are used as in the example of Fig 3, the two streams may need merging back together at a later time. Figure 4 illustrates this idea.

Version control tools provide support for this complex task of merging variants, although of course it is only possible with text (e.g. ASCII) files. However, the longer development streams remain apart, the more difficult the task of merging becomes. Of course, a rewrite of the CI could eliminate the need for merging, and, in the case of binary CIs, is the only choice.

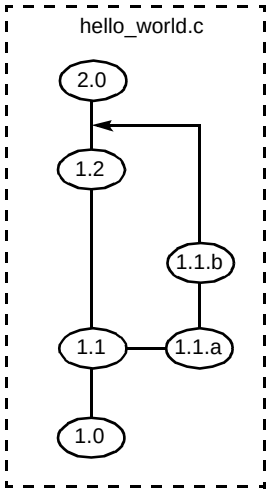

Fig 4 Merging CI variants.

4.7 Baselines revisited

Most version control tools manage the maintenance of baselines. All CI revisions forming baselines are in the version control system, and provided only the tool administrator or version manager can create and modify baselines, their integrity can be assured as far as humanly possible.

Figure 5 illustrates how baselines could be applied to CIs in a version control system. The baseline is created by applying tags to the CI revisions in the archive files.

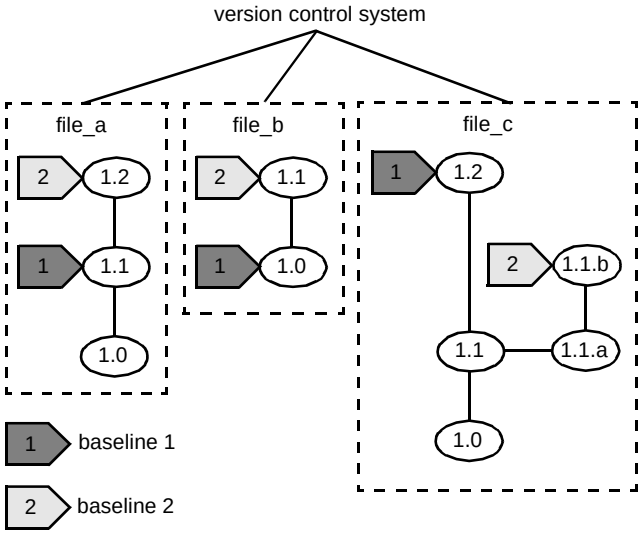

Fig 5 Baselines in the version control system.

Including all documentation (specifications, third party product lists, build documentation, etc) in the baseline allows the entire environment to be reconstructed to the state it was in when the baseline was taken. If every CI is in

the version control system, the tool can be told to find every CI revision with a particular baseline label, making reconstruction straightforward.

4.8 Knowing what you've got

From experience, a lack of version control in the management of specifications can lead to inconsistencies in and between their related implementations. Changes to design documentation may not be reflected in their implementations, and vice versa. If baselines are not managed, this can result in not knowing what is being tested, or if any faults found in testing will still exist. In the worst case, where change and version control have been lacking and project management poor, contractual disputes have arisen over what the contract (essentially another CI) should actually contain, and what the customer should have received.

5. Change control

5.1 Beyond version control.

Version control is often regarded as the traditional flavour of software configuration management. While it is fundamental, alone it is certainly not sufficient. In the development of any product, but especially for software systems, the only guarantee is that the system will evolve from its initial specification, and that problems will be found throughout its life cycle. Change control is the process whereby changes arising through a product's life cycle, whether enhancements or bug fixes, are initiated, progressed and managed.

Failure to manage change, associated with poor version control, may result in any number of problems:

- bugs mysteriously reappearing (poor version control resulting in the simultaneous update problem),
- changes being applied to the wrong CI revisions — these changes may become difficult to move if further changes have subsequently been applied,
- loss of previous CI revisions,
- synchronisation problems between components (e.g. sub-systems, requirements, specifications, tests, problem reports) — frequently caused by component owners not being kept informed of changes.

Change management allows configurations to be related to a higher level than CI versions, possibly even to the highest organisational level — a change in business objectives could be tracked right down to the CIs implementing that requirement, and vice versa.

5.2 Task-based CM

The 'state-of-the-art' in software configuration management is often considered to be the file-based version control explained above [4—6]. In this environment, software products are defined, modified, built and tested in terms of a collection of version files.

Working in a file by file method means individual changes cannot usually be related; but changes are rarely restricted to just one CI, and several CIs must be modified to make a single change. This information must be gathered, and communicated to those who need to know — developers, version managers, testers, release managers, etc. The release manager would be told which CIs and revisions are to be included in builds and releases. Often this means insufficient, incomplete or incompatible sets of changes are included.

At best the build will fail, or the fault will be discovered in integration testing. At worst, the fault will slip through manifesting itself as a failure at a customer site.

Humans do not operate well in terms of individual file versions, but would rather think of logical changes or 'tasks'. The checking out of files should be the only CM operation performed at the file level, and then preferably working up from a baseline.

In an environment integrating version and change control, baselines are no longer defined by CI versions, but as the previous baseline plus (or minus if rolling back) a number of changes, for example:

$$\text{Baseline } \Psi = \text{Baseline } \Phi + \text{Change } \rho + \text{Change } \sigma + \text{Change } \tau.$$

Such task-based systems are more intuitive and of greater integrity than file-based ones. Far more is known about the built product than just a list of file revisions; a list of enhancements, not files, defines the release. In file-based systems, failures arise as CIs implementing parts of changes are forgotten during integration, or as mistakes are made applying baselines and extracting CIs for building.

5.3 The change document

Change control provides a formal mechanism for raising, evaluating, approving, implementing and testing change requests (CRs) and problem reports (PRs). CRs are for functional enhancements, while PRs are for reporting problems such as specification non-compliances, bugs, performance and usability issues, etc.

CRs and PRs are structured forms, which are updated as the request for a change or fix is progressed. In all but the most trivial of projects, the information will need to be widely communicated — to management, developers,

testers, release managers, etc. Form and format are critical because control cannot exist without communication.

The data collected on the form must be sufficient to explain, for example, why the change is required, its impact, which CIs are affected, and who requested the change. Next, information must be gathered describing the problem in sufficient detail to allow a course of action to be recommended. Further data must be gathered as the CR/PR is implemented (e.g. CI revisions implementing the change), integrated (e.g. build number) and tested. Several CRs and PRs may be related, and the form must be able to manage this information.

The format of the form should remain simple and, ideally, consistent across the business. This is especially important where reuse is common, and components are shared widely. A familiar format will improve efficiency and eliminate confusion when the changes are assessed.

5.4 The change life cycle

After change requests and problem reports are raised they pass through a number of defined life cycle phases. Although it is reasonable to accept that different types of CI could be managed differently (e.g. requirements documents versus code modules), the number of different life cycle models through which the forms could progress should be kept to a minimum.

The generic life cycle model offered for project support by SI's Configuration Management Unit is illustrated in Fig 6.

After the request is raised, a technical evaluation is performed, with the aim of identifying the problem and the CIs responsible. It is then reviewed for severity, priority and impact upon the project by a Change Control Board (CCB). It is the CCB's responsibility to approve, monitor and control the conversion of design objects into system CIs and authorise changes to that system.

At this stage in the change life cycle the CCB may:

- decide to implement the request, and scope its inclusion against a release,
- request further technical evaluation,
- perform more detailed impact analysis, of both implementing and not implementing the change,
- reject the request — for example a CR outside the scope of the product's requirements,
- provide a work-round not requiring any changes.

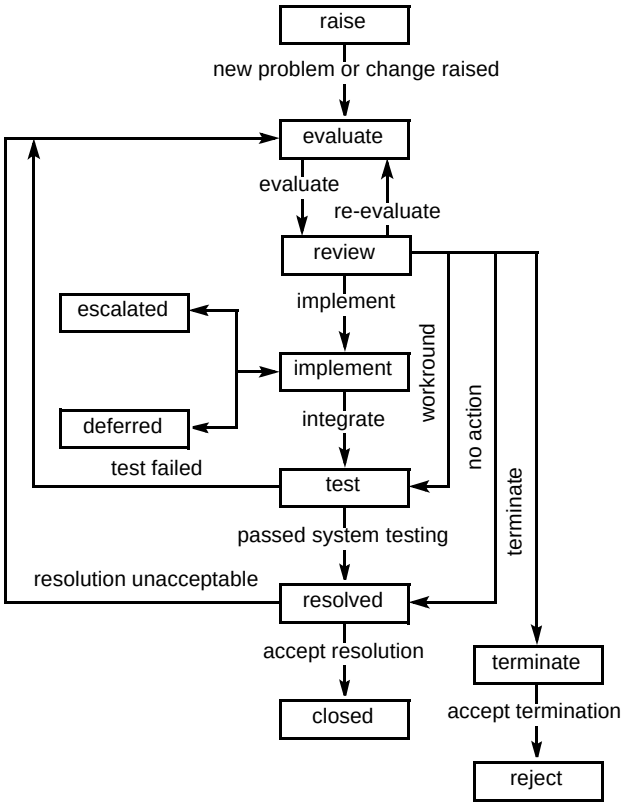

Fig 6 A typical change request life cycle.

Those requests having a serious impact on the product or customer may cause the change to be escalated, while low-priority changes may be deferred to later releases.

Changes then follow the implement, unit-test, integrate, integration-test cycle until they can be closed. If implementation is not successful, the request may need re-evaluation, for example, a fault's cause may not have been where expected.

The change control process allows information to be gathered for the production of reports and metrics. One of their uses is in identifying deficiencies in the development process and to trigger managerial action. For example, frequent changes to a component may indicate a specification or design weakness, while large numbers of integration problems would suggest a lack of unit testing.

As with version control, tool support is available and should be used for change control. The tools manage the progression of the change forms through their life cycle, automatically notifying those responsible for changes against CIs at each stage. In addition they manage the scoping of requests against builds and releases, and aid the production of metrics.

5.5 Linking versions and change

All effective CM systems must closely integrate the disciplines of version control and change control. The first reason is to relate CI changes to CRs and PRs, and the second to enforce the version control system's access controls.

Figure 7 illustrates the cross-referencing between CI revisions in the version control system, and CRs/PRs in the change control system. The link between the systems means the versions of CIs implementing a change are automatically recorded against the CR or PR. Conversely, the change log associated with the version control system's CI also contains a cross-reference to the CR or PR that resulted in the change.

Technical people characteristically have a desire to optimise and improve. While their intent is not malicious, the consequences of their changes can be far-reaching, counterproductive and difficult to trace. If developers can be persuaded only to make testable authorised changes, the exact configuration of a product is easily determined.

In the very early stages of development, access controls on CIs are often limited to allowing only one person to have write-access to a revision at a time. As projects mature and become more stable, access restrictions can be tightened. Changes are allocated owners, who are determined by the affected CIs. The CM system permits only the change owners (or delegated owners) to check out affected CIs, and only while the PR or CR is in an appropriate state, i.e. when it has been authorised for implementation by the CCB and before it has been unit tested.

While this is sufficient for low-level projects, the situation is more difficult across programmes and where external suppliers are used. Maintaining audit trails can be very difficult when change requests raised in one programme's CM system require changes to CIs in other independent systems. Although BT has an obligation to use only suppliers conforming to ISO 9000, it is not always the case that they can demonstrate the use of any CM practices. Hence the configuration management of external deliverables is commonly a particular cause for concern.

6. Build management

6.1 Selection of CIs for building

After the CRs and PRs scoped for a release have been implemented, a build must be performed to integrate the changes into the product. In CM, a build is defined as an operation that takes one or more CIs and performs some action on them to create new deliverable CIs. For software, this is the compilation and integration process.

At the time of the build, it is likely that several versions and variants of the CIs will exist. Selection of the correct CIs is critical to success. If the correct objects are not built (i.e. the wrong baseline was taken, or CIs not in the baseline are included, or the baseline was incorrect) then the value of the CM system is greatly reduced. However, coupling version control and change control provides a position where all baselines, builds, testing and releases can be defined in terms of a baseline plus CRs and PRs.

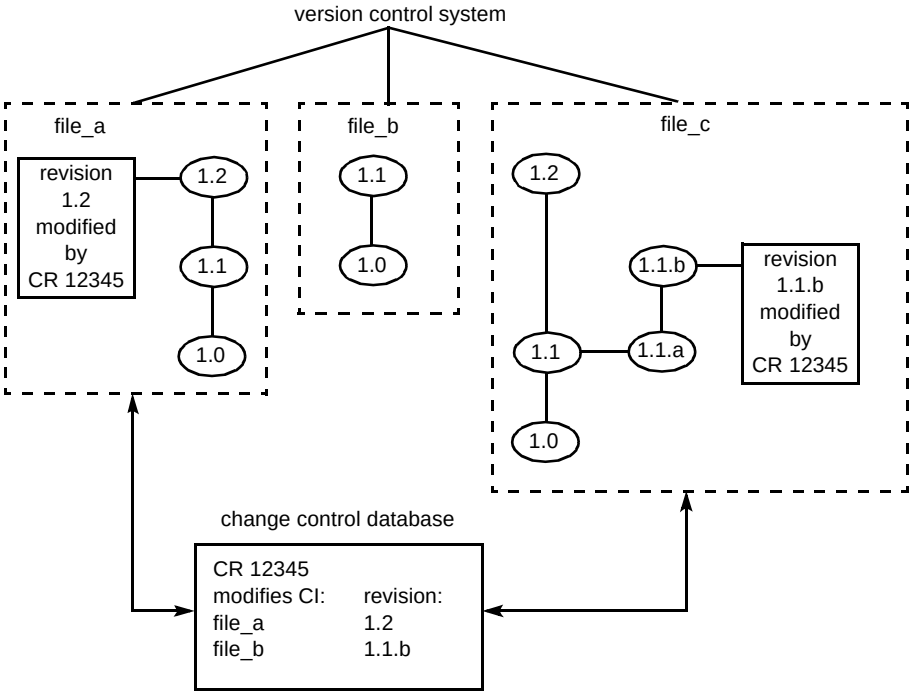

Fig 7 Integrating version control and change control.

It is critical that the sets of CRs and PRs included in builds are complete. For instance, if the CRs and PRs for a build referenced only revisions 1.4 and 1.6 of a particular CI (1.3 being in the previous baseline), what has happened to the CR/PR relating to revision 1.5? Problems could arise if the change for 1.5 (and hence probably 1.6) has dependencies in other CIs missed out of the build. The CRs and PRs must also be consistent, both with the previous baseline and with each other, e.g. variant revisions of the same CI cannot occur in the same build.

6.2 The build of independence

Depending upon the tools and CIs, before building the CIs may need extracting from the library, or they may be able to be built *in situ*. Software system builds usually follow a policy of minimal rebuilding, reusing as many derived CIs as possible. Only CIs with dependency changes must be rebuilt. Whatever methods are used, all builds must be performed independently from the development and test environments.

A primary objective of all CM systems is to ensure reproducibility. This enables later maintenance releases to vary from the base system in a well-defined manner and prevents the need for costly redevelopment.

Independent builds ensure systems are reproducible, with no features dependent upon the environments in which they were developed. These development environments cannot be assumed to conform to the baseline. For example, problems often occur when client/server systems are developed against databases whose contents are unknown, or when incorrect versions of third party products are used. Independent builds detect these problems, preventing customers from receiving defective installations.

6.3 Regression versus progression

Although not immediately apparent, build strategies have a significant impact upon the quality and timeliness of product releases.

As deadlines approach, the temptation to skimp on CM and quality grows. The objective is to save time and make as many changes as possible. The result will be the wrong fixes and fixes that are wrong being delivered.

Imagine a scenario where a release deadline is rapidly approaching. The build start date has passed and not all the changes scoped for release have been implemented. To save time, unit testing is not as thorough as usual; but the build fails. The deadline looms ever closer, resulting in a panic reaction from the project. The second attempt at the fixes are rushed, and not implemented correctly. It builds, but fails integration testing, which is also suffering. The release becomes later and later. The product's quality spirals rapidly downward to below acceptable standards, and a great deal of effort is wasted building and testing. Although this example sounds alarmist, it is one seen too often by configuration managers, and can only encourage software regression not progression.

Even when all CM procedures are followed, attempting to integrate too many changes in one release is almost guaranteed to cause problems. This is especially true where the changes are mostly unrelated. Rather than having one massive integration effort before release, changes should be built and integrated incrementally on top of earlier changes. For example, if a monthly release strategy is being followed, a weekly build and test cycle could be used. Continuous integration:

- helps pinpoint the sources of incompatibilities earlier, because less has changed since the previous build,
- increases the amount of testing each enhancement receives through extra regression testing and hence reduces the number of releases,
- shortens the time to market, as a single incremental fix requires much less testing,
- results in the release to the customer being of higher quality, enhancing BT's image.

6.4 Build automation

As with many product assemblies, large software systems have many components to be built and integrated. It is vital the CIs are built using the correct tools and that those tools are used appropriately.

Automation of the build process is one solution. While this is usually relatively straightforward for server (e.g. Unix) builds, the automation of PC client builds is non-trivial. However, it is an area in which BT's Systems Integration unit has recently made significant advances, resulting in notable efficiency improvements and cost savings.

Builds come in two flavours —full releases and patches. Patches are partial support releases to fix problems ahead of the next full release. As their name suggests, patches should be the exception to the rule. In far too many instances this is not the case, impacting the value of automating builds. Two patch builds are rarely identical, and automating becomes a large overhead adding very little value, apart from in aiding reproduction.

Continuous integration build strategies should encourage automation. The result is that changes are more thoroughly tested and hence requiring less patching and fewer maintenance releases. Automation becomes cost

efficient, and the automation scripting becomes part of the baseline, further enhancing reproducibility.

6.5 Next steps

To ensure product integrity, testing must be independent from development, and must only accept releases through the authorised channel, i.e. from the independent build environment. The changes to the product are communicated to the testers through the change control system. All test cases should be subject to CM control.

Until a product comes to the end of its useful life, the whole cycle of raising faults, requesting enhancements, updating CIs, building, testing, raising faults, reworking and releasing will be repeated many times.

6.6 Release management

After a product has been built and passed testing (i.e. the ISO 9000 Quality Gate 10 criteria) it is finally released. In effect, the point of release is the only place in the life cycle where all components comprising a product come together (e.g. software, user guides, training courses, marketing campaign). All releases must be accompanied by release documentation, detailing, as a minimum, the release identifier, CRs and PRs fixed, any known faults outstanding, the installation environment, and installation instructions.

As with build management, a release strategy and plan must be developed. The basic strategic options are timed releases, or releasing as defined faults are fixed and changes implemented. The route taken must take into account the cost of making the release against its value to the customer and BT.

Release management provides a mechanism for recording which customers have which installations (e.g. hardware, software, and third-party products). As more variants are created and reuse increases, release management becomes essential in product support. Compatibility matrices, detailing which versions of products work together, become critical and play a role in the scheduling of future releases.

Changes must never be applied directly to live systems, however exceptional the situation. When this happens there is no guarantee the system can be reconstructed, and often no records of the change are kept. Furthermore, no testing can be performed before the live system is affected, or more likely, infected. In short, the consequences of uncontrolled change can be disastrous. Although the need to quickly implement, test and release emergency bug fixes will always be present (all perhaps within a day), the CM system must be efficient enough not to be bypassed for such critical activities.

7. Implementing configuration management

7.1 Evolution not revolution

Introducing CM is a complex multi-dimensional problem, with technical, political, organisational and cultural issues. So how should it be approached?

The top-down, business-process re-engineering approach is considered but this takes a significant time to develop and much effort to understand, correct and document. It can also result in unworkable solutions. The alternative bottom-up method is quick with little everyday interference. Yet it has no formal strategy and no time to assess the current processes and the overall picture. Pockets of expertise grow, but rarely sufficiently, the implementers not having sufficient visibility or influence. The result is a lack of consistency across the product and business.

The implementation of CM should be led by considering the development processes in use. Why are things done? Have processes evolved, and carried on being used through habit, to avoid limitations or restrictions that no longer apply? The introduction of CM drives change in the processes. There will be elements of risk involved, primarily in the people, technical and organisational arenas, but it offers the chance to reduce overall risk in the development process [7].

7.2 People risks

There are many different understandings of CM's scope due to the wide range of its definitions. On one hand, buying a tool is often expected to magically solve all CM problems. On the other is a fear of bureaucracy and 'big brother' management with too much visibility and control.

Individuals may see CM as costing time and diminishing their own productivity, especially when schedules are tight. Quality may be sacrificed, and meeting release deadlines can become the only priority. An individual's tunnel vision may not allow them to see improved group productivity [8]. They may appear to have high productivity, but if it is uncontrolled their colleagues may spend all their time adapting to the individual's changes. The entire team's efforts must be devoted to achieving the scoped work.

The role of configuration manager is not always easy, and can provoke unpopularity as procedures are enforced. Process is not, or need not be, the same as wasteful bureaucracy. Bad processes add overheads and do not increase quality sufficiently to be cost effective. Good processes organise large, complex and vague tasks into a series of well-defined and manageable ones.

Besides issues of consistency and reuse, the threat of taking short cuts due to managerial and peer pressure is a strong argument for keeping CM independent from the development and maintenance environments.

For a CM system to work, everyone must be convinced of its value. CM must not be seen as an overhead, but as the normal method of working which reduces effort — an integral part of the IT and quality strategies.

7.3 Technical risks

A major risk when introducing CM is an inability to describe applications' architectures and hence their interdependencies. Variants need to be supported, and 'monolithic' code structures may require re-engineering for maintainability. Such risks only exist because CM is lacking.

No CM process can anticipate every eventuality, so an effective system must be flexible, while always maintaining an audit trail for accountability.

7.4 Organisational risk

There is a real risk that a particular CM tool will become the governor of change rather than the CM process. Before tools are adopted, the environment in which they are needed must first be assessed, then decisions can be made regarding which tools are appropriate and how they should be used. For example, all good version control tools will allow large numbers of variants to be created. Without a tool, this would have been beyond comprehension. Just because something is possible does not mean it should be done.

Security issues may cause problems. For instance government security requirements prohibiting the exchange of information could prevent the use of centralised distributed CM systems. In such systems, the control of information in the change management database may not be strict enough, and shared version control libraries unacceptable. This in turn could hinder reuse, and lead to the multiple-maintenance problem.

Constant reorganisation within the company destabilises resource availability and stability, hence the argument for a centralised CM authority to provide continuity. It is impossible to over-emphasise the importance of stability to successful software development.

Implementing CM depends upon finding the balance between chaos and bureaucracy. Control must be tight enough to allow development teams to be devoted to productive work, and loose enough for exactly the same reason. If control is too tight, time is wasted fighting bureaucracy. If it is too loose, what have you got anyway?

8. The future of CM within BT

At the time of writing, while there are many examples of good CM practice across BT, this is not always the case. Given our commitment to quality, especially in light of the ISO 9000 requirements, it is essential CM becomes an accepted, everyday activity within our software life cycles.

Therefore, efficient and acceptable CM processes must be put in place across the business. However, as previously mentioned, individual pockets of CM cannot grow sufficiently to encompass the whole business. SI's Configuration Management unit offers a generic CM support infrastructure that should permit consistency across BT.

Going beyond the 'low-level' CM described in this paper, SI's Enterprise Configuration Management Programme aims to provide visibility of higher level CM problems across the business as a whole, e.g. software system compatibility and dependencies; for further details, see Moor et al [9]. However, due to the number and size of products BT manages, this will take considerable time, effort and commitment on all parts to achieve.

9. Conclusions

The intangible nature and susceptibility to change that characterise software make it difficult to control. Changes to software can be meaningless in the absence of reference points, and must be carefully managed. Version control is fundamental for recording a product's evolution in relation to change, and for ensuring the reproducibility and maintainability of any configuration past or present. Configuration management systems must be integrated, independent, and be able to impose control without bureaucracy.

Configuration management helps deliver highly functional quality software, on time and to budget, and helps with the development, support and maintenance tasks in the longer term. It has the side effect that the software development process improves and comes under control, resulting in further productivity gains.

CM is essential to a mature development procedure: its goal is not perceived short-term individual productivity, but real long-term team productivity. If a product is delivered late with good CM, how late would it be with no CM?

Acknowledgements

The author wishes to thank the friends and colleagues who have encouraged and supported the writing of this paper, contributing valuable insights into the world of software engineering.

References

1 Samaras T T: 'Configuration management deskbook', Advanced Applications Consultants Inc (1988).

2 Berlack H R: 'Software configuration management', John Wiley and Sons Inc (1992).

3 'Ovum Evaluates: Configuration Management Tools', Ovum Ltd (1995).

4 Mitchell R J: 'Managing Complexity in Software Engineering' Peter Peregrinus Ltd (on behalf of the IEE) (1990).

5 Tichy W F (Ed): 'Trends in Software: Configuration Management', John Wiley & Sons Inc (1994).

6 Cagen C and Wiborg-Weber D: 'Task based configuration management', Continuus Software Corporation, http://www.continuus.com:80/developers/developersACEA.html (November 1995).

7 Dart S and Krasnov J: 'Experiences in risk mitigation with configuration management', 4th SEI Risk Conference, Continuus Software Corporation, http://www.continuus.com:80/developersACE.html (November 1995).

8 Babich W A: 'Software configuration management — coordination for team productivity', Addison-Wesley, (1986).

9 Moor S R, Gunne-Braden J H and Kidd C: 'Enterprise CM — controlling integration complexity', BT Technol J, 15, No. 3, pp 61—72 (July 1997).

Stephen M Thompson joined BT in 1991 as a sponsored student. After graduating from the University of York in 1995 with an MEng (Hons) degree in Computer Systems and Software Engineering he joined Systems Integration's Configuration Management unit.

Since then he has developed tools for automating the PC client build process, managed software builds for the SDH programme, and been involved in the PCD call centre expansion programme as a CM consultant. He now works in Applied Research and Technologies' Advanced Media Unit.